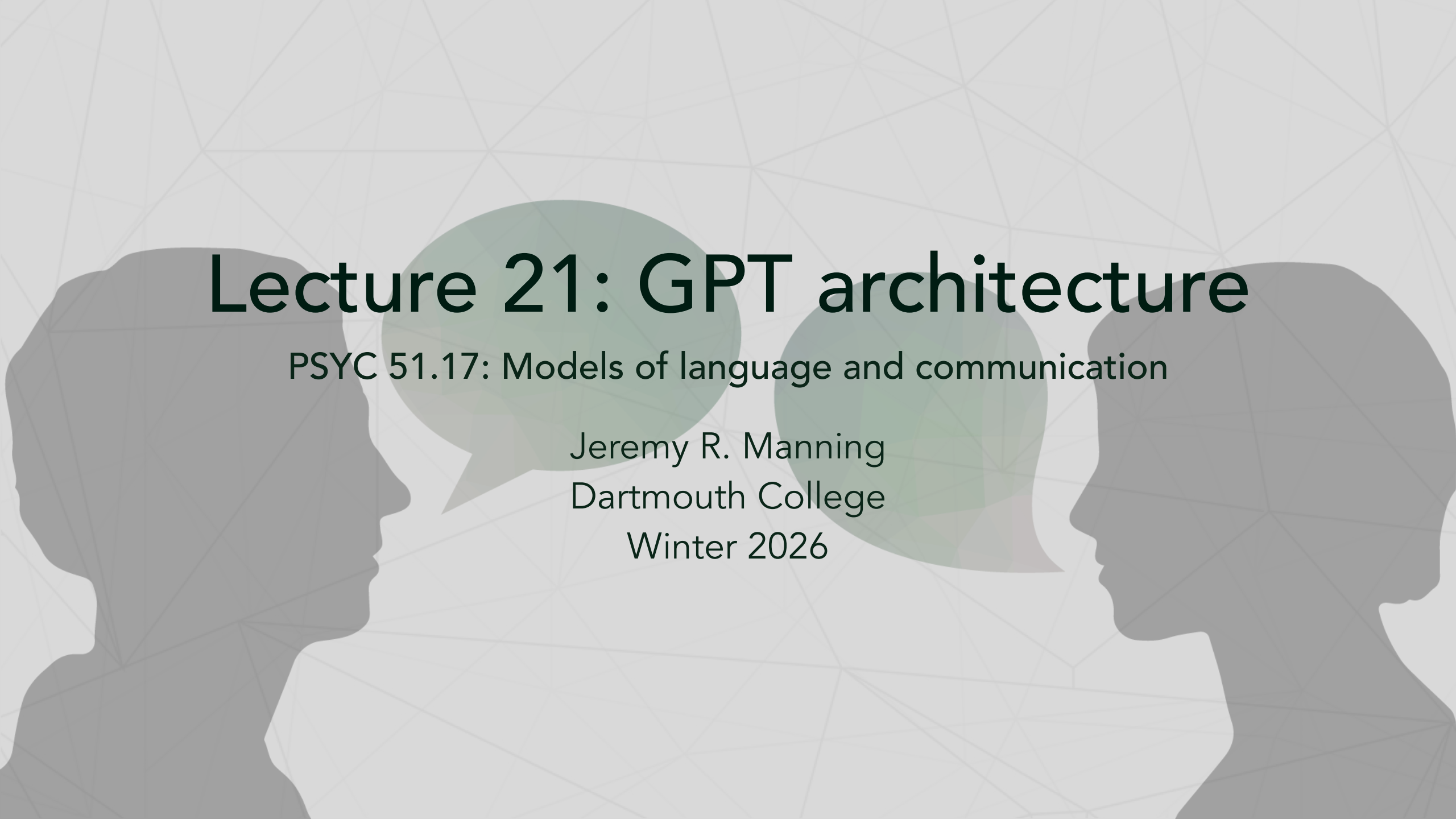




Lecture 21: GPT architecture

PSYC 51.17: Models of language and communication

Jeremy R. Manning
Dartmouth College
Winter 2026

Learning objectives

By the end of this lecture, you will be able to...

- Explain the **generative pre-training** paradigm and why it was revolutionary
- Describe the **transformer decoder** architecture used in GPT
- Implement **masked (causal) self-attention** and explain why it is needed
- Distinguish between **pre-training** and **fine-tuning** stages
- Compare GPT's **autoregressive** approach with BERT's **bidirectional** approach
- Understand GPT-1's strengths, limitations, and historical significance

Announcements

Assignment 4 due this week

Assignment 4 (Customer Service Chatbot) is due **February 16, 11:59 PM EST**.

Final project and optional assignment

- **Final Project** has been released! Due **March 9, 11:59 PM EST**
- **Assignment 5** (Build GPT) is now available as **optional extra credit**

From BERT to GPT

Two philosophies of language modeling

- **BERT** (encoder): Mask random tokens, predict them using **bidirectional** context. Best for *understanding* tasks (classification, NER, QA).
- **GPT** (decoder): Predict the **next token** using only **left-to-right** context. Natural for *generation* tasks, but also capable of understanding.

Questions to consider

Both models were published in 2018. BERT initially dominated benchmarks, but GPT's approach proved more scalable. Why might autoregressive modeling have a higher ceiling?

The generative pre-training paradigm

Radford et al. (2018): 'Improving Language Understanding by Generative Pre-Training'

GPT introduced a two-stage approach that changed NLP forever:

1. **Pre-train** on massive unlabeled text (unsupervised)
2. **Fine-tune** on specific downstream tasks (supervised)

Why this was revolutionary

- **Before GPT:** Train task-specific models from scratch, needing large labeled datasets for each task
- **After GPT:** Learn general language representations once, then adapt cheaply to any task
- This is **transfer learning** for NLP -- the same idea that transformed computer vision with ImageNet

The transformer decoder

GPT's architecture

GPT uses only the **decoder** half of the original transformer (Vaswani et al., 2017). The data flows through:

1. **Token embeddings**: Map each token to a 768-dimensional vector
2. **Position embeddings**: Add learned positional information
3. **N transformer blocks**: Each contains masked self-attention + feed-forward network
4. **Output head**: Project back to vocabulary size for next-token prediction

Key difference from the full transformer

The original transformer has both an encoder and a decoder with cross-attention between them. GPT removes the encoder entirely and removes cross-attention -- it only uses **masked self-attention** within the decoder.

Masked (causal) self-attention

The core mechanism

In standard self-attention, every token attends to every other token. In **causal** (masked) self-attention, each token can only attend to tokens at **earlier positions** (and itself). This prevents the model from "peeking" at future tokens during training.

Attention pattern for 'The cat sat on'

	The	cat	sat	on
The	1.0	-inf	-inf	-inf
cat	0.3	0.7	-inf	-inf
sat	0.2	0.3	0.5	-inf
on	0.1	0.2	0.3	0.4

The **-inf** entries become 0 after softmax, ensuring no information flows from future tokens.

Why causal masking matters

Three benefits of the causal mask

1. **Autoregressive property:** The model generates text left-to-right, one token at a time. Each prediction depends only on previous tokens -- exactly matching how generation works at inference time.
2. **No information leakage:** During training, the model cannot cheat by looking ahead at the answer. This forces it to learn genuine predictive representations.
3. **Parallel training:** Despite the left-to-right constraint, all positions can be trained simultaneously using the mask. The model computes predictions for every position in one forward pass.

Contrast with BERT

BERT's masked language model sees context in *both* directions but can only predict ~15% of tokens per pass. GPT predicts *every* token per pass but sees context in

Implementing the causal mask

Creating a causal mask in PyTorch

```
1 import torch
2
3 def create_causal_mask(seq_len):
4     """Create lower-triangular mask for causal attention."""
5     mask = torch.ones(seq_len, seq_len)
6     mask = torch.tril(mask) # Keep lower triangle (including
7                             diagonal)
8     return mask
9
10 mask = create_causal_mask(4)
11 # tensor([[1., 0., 0., 0.],
12 #          [1., 1., 0., 0.],
13 #          [1., 1., 1., 0.],
14 #          [1., 1., 1., 1.]])
```

How the mask is applied

Position embeddings

Why position information is needed

Self-attention is **permutation invariant** -- it treats tokens as a *set*, not a *sequence*. Without position information, "The cat sat on the mat" and "mat the on sat cat The" would produce identical representations.

GPT's approach: learned position embeddings

- Each position (0 to max_length - 1) gets a **learnable** embedding vector
- Added element-wise to token embeddings: `embedding = token_emb + pos_emb`
- Maximum sequence length: **512 tokens** (GPT-1)
- The model learns position patterns during training (e.g., that position 0 is often a sentence start)

Alternative: sinusoidal encodings

The original transformer used fixed sinusoidal functions. GPT chose learned embeddings for greater flexibility, though this limits generalization to unseen sequence lengths.

GPT embedding layer in code

Token + position embeddings

```
1 import torch
2 import torch.nn as nn
3
4 class GPTEmbedding(nn.Module):
5     def __init__(self, vocab_size, d_model, max_seq_len):
6         super().__init__()
7         self.token_embed = nn.Embedding(vocab_size, d_model)
8         self.pos_embed = nn.Embedding(max_seq_len, d_model)
9
10    def forward(self, x):
11        seq_len = x.size(1)
12        positions = torch.arange(seq_len, device=x.device)
13        return self.token_embed(x) + self.pos_embed(positions)
14
15 # Usage
```

continued...

GPT embedding layer in code

Token + position embeddings

```
16 embed = GPTEmbedding(vocab_size=40000, d_model=768,  
17 max_seq_len=512)  
17 tokens = torch.tensor([[101, 2054, 2003]]) # "The cat sat"  
18 embeddings = embed(tokens) # Shape: (1, 3, 768)
```

...continued

The GPT transformer block

Pre-norm transformer block

Each GPT block applies two sub-layers with **residual connections** and **layer normalization**:

1. **LayerNorm** → **Masked multi-head attention** → **Add residual**
2. **LayerNorm** → **Feed-forward network** (expand 4x, GELU, project back) → **Add residual**

GPT block implementation

```
1 class GPTBlock(nn.Module):
2     def __init__(self, d_model, n_heads):
3         super().__init__()
4         self.norm1 = nn.LayerNorm(d_model)
5         self.attention = MaskedMultiHeadAttention(d_model,
6             n_heads)
7         self.norm2 = nn.LayerNorm(d_model)
8         self.ffn = nn.Sequential(
9             nn.Linear(d_model, 4 * d_model),
```

continued

The GPT transformer block

GPT block implementation

```
9         nn.GELU(), # GPT uses GELU, not ReLU
10         nn.Linear(4 * d_model, d_model)
11     )
12
13     def forward(self, x):
14         x = x + self.attention(self.norm1(x)) # Pre-norm +
residual
15         x = x + self.ffn(self.norm2(x))      # Pre-norm +
residual
16     return x
```

...continued

Complete GPT model

Putting it all together

```
1 class GPT(nn.Module):
2     def __init__(self, vocab_size, d_model, n_layers, n_heads,
3       max_len):
4         super().__init__()
5         self.token_embed = nn.Embedding(vocab_size, d_model)
6         self.pos_embed = nn.Embedding(max_len, d_model)
7         self.blocks = nn.ModuleList([
8             GPTBlock(d_model, n_heads) for _ in range(n_layers)
9         ])
10        self.norm = nn.LayerNorm(d_model)
11        self.head = nn.Linear(d_model, vocab_size)
12
13    def forward(self, x):
14        seq_len = x.size(1)
```

continued...

Complete GPT model

Putting it all together

```
14     positions = torch.arange(seq_len, device=x.device)
15     x = self.token_embed(x) + self.pos_embed(positions)
16     for block in self.blocks:
17         x = block(x)
18     x = self.norm(x)
19     return self.head(x)  # (batch, seq_len, vocab_size)
```

...continued

GPT-1 specifications

Model architecture

Component	Value
Transformer layers	12
Hidden size (d_model)	768
Attention heads	12
Max sequence length	512 tokens
Vocabulary size	40,000 (BPE)
Feed-forward size	3,072 (4 x 768)
Total parameters	~117 million

Training details

Detail	Value
Training data	BooksCorpus (~7 000 books)

Stage 1: Pre-training

The pre-training objective

Maximize the likelihood of each token given all preceding tokens:

$$\mathcal{L}_{\text{pre-train}} = \sum_{i=1}^N \log P(t_i \mid t_1, t_2, \dots, t_{i-1}; \Theta)$$

This is simply **next token prediction** over a large corpus. No labels needed.

Training on 'The quick brown fox jumps'

The model learns from every position simultaneously:

- Input: "The" → Target: "quick"
- Input: "The quick" → Target: "brown"
- Input: "The quick brown" → Target: "fox"
- Input: "The quick brown fox" → Target: "jumps"

Every text passage provides training signal at every token position.

Stage 2: Fine-tuning

Adapting to downstream tasks

After pre-training, GPT is adapted to specific tasks by:

1. **Reformatting** task inputs with special delimiter tokens
2. **Adding** a small classification head (linear layer) on top
3. **Training** with both task loss and a language modeling auxiliary loss

$$\mathcal{L}_{\text{fine-tune}} = \mathcal{L}_{\text{task}} + \lambda \cdot \mathcal{L}_{\text{LM}}$$

The auxiliary LM loss ($\lambda = 0.5$) improves generalization and speeds convergence.

Fine-tuning hyperparameters

- **Learning rate:** 6.25e-5 (much lower than pre-training!)
- **Batch size:** 32
- **Epochs:** 3
- **LR schedule:** Linear warmup then linear decay to 0

Task formatting for fine-tuning

How GPT handles different tasks

All tasks are converted to a sequence format using special delimiter tokens:

Task	Input format
Classification	[START] text [DELIM] → predict label
Entailment	[START] premise [DELIM] hypothesis [DELIM] → predict relation
Similarity	[START] text_A [DELIM] text_B [DELIM] → predict score
QA / Reading comprehension	[START] context [DELIM] question [DELIM] → predict answer

The unifying insight

Every NLP task can be reformulated as **text completion**. The model reads a formatted input and produces an output -- the same thing it learned to do during pre-training. This is why pre-training transfers so effectively.

Fine-tuning example: sentiment classification

Classifying movie reviews

```
1  # Format input for GPT
2  text = "This movie was absolutely fantastic!"
3  formatted = f"[START] {text} [DELIM]"
4
5  # Tokenize and pass through GPT
6  token_ids = tokenizer.encode(formatted)
7  hidden_states = gpt_model(token_ids) # (1, seq_len, 768)
8
9  # Take hidden state at the last (DELIM) position
10 final_hidden = hidden_states[0, -1, :] # (768,)
11
12 # Pass through classification head
13 classification_head = nn.Linear(768, 2) # 2 classes
14 logits = classification_head(final_hidden) # [pos_score,
15 neg_score]
```

continued...

Fine-tuning example: sentiment classification

Classifying movie reviews

```
16 # Compute loss
17 loss = cross_entropy(logits, label_id) # label_id = 0 (positive)
```

...continued

What does GPT learn at each layer?

Progressive abstraction through layers

Layers	What is learned	Example
1--3	Word order, basic grammar	"The cat" vs. "cat The"
4--6	Syntactic structure	Subject-verb agreement
7--9	Semantic relationships	"bank" (river) vs. "bank" (financial)
10--12	World knowledge, reasoning	"Paris is the capital of ..." → "France"

Questions to consider

This progressive abstraction mirrors findings in both computer vision (Zeiler & Fergus, 2014) and neuroscience (Fedorenko et al., 2024). Why might hierarchical representations emerge across such different systems?

GPT-1 results

Performance on standard benchmarks

Task	Previous SOTA	GPT-1
Question answering	86.7	88.1
Semantic similarity	85.0	85.8
Text classification	93.0	94.2
Natural language inference	80.6	82.1

GPT-1 achieved state-of-the-art on **9 out of 12** benchmark tasks.

The key finding

The largest improvements came on tasks with **less training data**. Pre-training provided a strong prior that compensated for limited labeled examples -- exactly the promise of transfer learning.

Zero-shot and few-shot learning

Learning without (much) fine-tuning

- **Zero-shot:** No task-specific examples. The model must generalize purely from its pre-training.
- **Few-shot:** A small number of examples (1--10) provided as context in the prompt.
- **Fine-tuning:** Full supervised training on a task-specific dataset.

GPT-1's limitation

GPT-1's zero-shot performance was **weak**. The model had learned rich language representations but struggled to apply them without explicit task formatting. This motivated the development of GPT-2 and GPT-3, which aimed to make models that could perform tasks *without* fine-tuning.

GPT vs. BERT: a comparison

Head-to-head comparison

Aspect	BERT (encoder)	GPT (decoder)
Training objective	Masked language model	Autoregressive LM
Context direction	Bidirectional	Left-to-right
Best suited for	Understanding (NER, QA, classification)	Generation (text, code, dialogue)
Fine-tuning approach	Task-specific heads	Unified sequence format
Base parameters	110M	117M
Tokens predicted per pass	~15% (masked only)	100% (every position)

Questions to consider

BERT dominated understanding benchmarks in 2018--2019, but GPT's autoregressive approach proved more scalable and versatile. The ability to *generate* text opened up entirely new capabilities that bidirectional models could not easily match.

Limitations of GPT-1

What GPT-1 could not do well

- **Weak zero-shot performance:** Required fine-tuning for each new task
- **Small model:** 117M parameters (small by modern standards)
- **Limited data:** Trained on ~5B tokens from BooksCorpus only
- **Short context:** Maximum sequence length of 512 tokens
- **Inconsistent generation:** Output could be incoherent over long passages
- **Hallucinations:** Confidently stated incorrect facts

The path forward

Each limitation suggested a clear direction: bigger models, more data, longer contexts, better training. GPT-2 and GPT-3 would systematically address these limitations through **scale**.

Key takeaways

Core concepts from this lecture

1. **Generative pre-training** introduced the pre-train → fine-tune paradigm for NLP
2. GPT uses a **transformer decoder** with **masked (causal) self-attention**
3. The causal mask ensures each token only attends to **previous** tokens
4. **Pre-training** learns general language patterns; **fine-tuning** adapts to specific tasks
5. All tasks can be reformulated as **text completion** with delimiter tokens
6. GPT-1 proved transfer learning works for NLP, but required fine-tuning for each task

Further reading

References

- **Radford et al. (2018)** -- "Improving Language Understanding by Generative Pre-Training" [[PDF](#)]
- **Vaswani et al. (2017)** -- "Attention is All You Need" [[arXiv](#)]
- **Devlin et al. (2018)** -- "BERT: Pre-training of Deep Bidirectional Transformers" [[arXiv](#)]
- **The Illustrated GPT-2** by Jay Alammar [[Blog](#)]

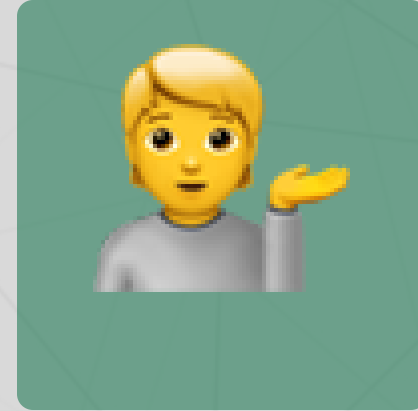
Questions?



Email



Discord



Office hours

Up next...

Scaling up: from GPT-1 to GPT-3, scaling laws, and the path to ChatGPT